



ENGIN 604 INTRODUCTION TO PYTHON FOR FINANCE

INTRODUCTION TO NUMPY - SOLUTION

Professor: *Gabriel E. Cabrera*
Teaching Assistant: *Alex Den Braber*



1 Arrays

Objects such as lists (`list`) are very convenient because of their flexibility (they are mutable); however, this kind of structure has a cost: it uses a lot of memory and its performance is poor. Financial and/or scientific applications require high-performance operations on special structures. One of the most important data structures is the *array*. *Arrays* usually organize other objects of the same data type both in rows and in columns.

If we assume that the most relevant data types are `float` and `int` (the idea can be extended to other data types), a one-dimensional **array** would represent, mathematically speaking, a row or column vector (depending on how it is defined). The most common cases are matrices, which would be a two-dimensional **array** $i \times j$ (row and column).

We therefore need a class capable of working efficiently with *arrays*. The solution is the **NumPy** library with its `ndarray` class. If we want to create the matrix X :

$$X = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

With lists (`list`):

```
x = list([[1,2,3],[4,5,6],[7,8,9]])  
x
```

```
## [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

Where each element nested in the list would be a row of the matrix X and each element of the nested element a column. If we want to see the value at position 2×2 :

```
x[1][1]
```

```
## 5
```

However, Python interprets the variable X as a list and not as an **array**. Using `numpy`

```
import numpy as np
```

```
mat_x = np.array(x) # the array is generated  
mat_x
```

```
## array([[1, 2, 3],  
##        [4, 5, 6],  
##        [7, 8, 9]])
```

Now the variable `mat_x` is an *array* and as such it has the following attributes:

- Dimensions:

```
mat_x.ndim # it has two dimensions
```

```
## 2
```

- The values of the dimensions:

```
m, n = mat_x.shape # number of rows and columns
m, n
```

```
## (3, 3)
```

- All the elements:

```
mat_x.size
```

```
## 9
```

- New class:

```
type(mat_x)
```

```
## <class 'numpy.ndarray'>
```

the `range()` version that is *built-in* in Python exists in **NumPy** as `arange()`.

```
np.arange(1,10).reshape(-1,3)
```

```
## array([[1, 2, 3],
##        [4, 5, 6],
##        [7, 8, 9]])
```

The library documentation can be found at <https://numpy.org/doc/stable/reference/>.

2 NumPy

1. Generate the following matrix:

$$A = \begin{pmatrix} 3 & 0 & 2 \\ 2 & 0 & 2 \\ 0 & 1 & 1 \end{pmatrix}$$

- a. Find the transpose of the matrix A .

```
num = [3, 0, 2, 2, 0, 2, 0, 1, 1]
```

```
list_to_array = np.array(num)
a = list_to_array.reshape((3,3))
```

```
# way 1
```

```
a.T
```

```
## array([[3, 2, 0],
##        [0, 0, 1],
##        [2, 2, 1]])
```

```
# way 2
```

```
a.transpose()
```

```
## array([[3, 2, 0],
##        [0, 0, 1],
##        [2, 2, 1]])
```

- b. Find the inverse matrix of A .

```
b = np.linalg.inv(a)
```

- c. Show that the following holds:

$$AB = BA = I$$

Where B is the inverse matrix of A and I the identity matrix.

```
a.dot(b)
```

```
## array([[1.0000000e+00, 4.4408921e-16, 0.0000000e+00],
##        [0.0000000e+00, 1.0000000e+00, 0.0000000e+00],
##        [0.0000000e+00, 0.0000000e+00, 1.0000000e+00]])
```

```
# to generate an identity matrix
```

```
np.eye(3)
```

```
## array([[1., 0., 0.],
##        [0., 1., 0.],
##        [0., 0., 1.]])
```

- d. Using matrices, compute the mean of A .

```
mean_a = np.ones((1,3)).dot(a).dot(np.ones((3,1))) / a.size
mean_a[0][0]
```

```
## np.float64(1.2222222222222223)
```

2. Generate the following matrices:

$$C = \begin{pmatrix} 2 & 4 \\ 5 & -6 \end{pmatrix} \quad \text{and} \quad D = \begin{pmatrix} 9 & -3 \\ 3 & 6 \end{pmatrix}$$

- a. Obtain the sum of C and D .

```
c = np.array([[2,4],[5,-6]])
d = np.array([[9,-3],[3,6]])
```

```
c + d # addition
```

```
## array([[11,  1],
##        [ 8,  0]])
```

```
c - d # subtraction
```

```
## array([[ -7,  7],
##        [  2, -12]])
```

```
c * d # multiplication
```

```
## array([[ 18, -12],
##        [ 15, -36]])
```

```
c / d # division
```

```
## array([[ 0.22222222, -1.33333333],
##        [ 1.66666667, -1.          ]])
```

- b. Obtain the dot product of C and D .

```
c.dot(d)
```

```
## array([[ 30,  18],
##        [ 27, -51]])
```

- Write a function that multiplies (not the dot product) two matrices.

```
mat_a = np.array([[1,2,3],[4,5,6]])
mat_b = np.array([[1,2,3],[4,5,6]])

def manual_mult(mat_a, mat_b):
    '''
    multiplies element ij of matrix a with element ij of matrix b
    '''
    m,n = mat_a.shape

    zero_mat = np.zeros((m,n))

    for i in range(m):
        for j in range(n):
            zero_mat[i][j] = mat_a[i][j] * mat_b[i][j]

    return(zero_mat)

# we test the function
manual_mult(mat_a, mat_b)

## array([[ 1.,  4.,  9.],
##        [16., 25., 36.]])
```

3 Random Numbers

NumPy has its own pseudo-random number generator (Mersenne Twister). The generator is a mathematical function that produces a sequence of “random numbers”. It takes a parameter to start the sequence of numbers, called the seed (*seed*). The function is deterministic, that is, with the same seed it will always produce the same sequence of numbers (the choice of the seed does not matter).

- Generate 10000 values from a standard normal distribution $\sim N(\mu = 0, \sigma = 1)$.
 - From the values created, generate a matrix E whose dimensions are 100×100 . What is the difference between `reshape()` and `resize()`?

```
np.random.seed(10)

array_norm = np.random.randn(10000)

# reshape
e = array_norm.reshape((100,100))

# resize
array_norm.resize((100,100), refcheck=False)
array_norm # in-place, but it must not have been referenced, for example with reshape
```

- “Flatten” the matrix E and verify that the data are indeed distributed with $\mu = 0$ and $\sigma = 0$.

```
e_flatten_col = e.flatten(order='C') # flattened by column
e_flatten_row = e.flatten(order='F') # flattened by row

e_flatten_col.mean() # mean
e_flatten_col.std() # standard deviation

array_norm
```

4 System of Equations

1. Use matrices to solve the following systems of equations:

a.

$$\begin{aligned}a + b + c &= 6 \\ 3a - 2b + c &= 2 \\ 2a + b - c &= 1\end{aligned}$$

```
# the matrices are generated (unknowns + result)
mat1 = np.array([[1, 1, 1], [3, -2, 1], [2, 1, -1]])
mat1_res = np.array([6, 2, 1])
```

```
# the system of equations is solved
np.linalg.solve(mat1, mat1_res)
```

```
## array([1., 2., 3.])
```

b.

$$\begin{aligned}3a + 4b - 5c + d &= 10 \\ 2a + 2b + 2c - d &= 5 \\ a - b + 5c - 5d &= 7 \\ 5a + d &= 4\end{aligned}$$

```
# the matrices are generated (unknowns + result)
mat2 = np.array([[3, 4, -5, 1], [2, 2, 2, -1], [1, -1, 5, -5], [5, 0, 0, 1]])
mat2_res = np.array([10, 5, 7, 4])
```

```
# the system of equations is solved
np.linalg.solve(mat2, mat2_res)
```

```
## array([ 1.24778761,  1.01769912, -0.88495575, -2.23893805])
```